

Actividad 4: Implementación de Logging & Monitoring

Tema: Registro y monitoreo de eventos de seguridad

Objetivo: Implementar un sistema de logging y detección de eventos sospechosos

¿Qué es Logging & Monitoring?

Es el proceso de **registrar y analizar eventos de seguridad** en servidores y aplicaciones para **detectar y responder a amenazas**.

Configurar logs de eventos en Apache

Modificar el archivo: `/etc/apache2/apache2.conf`

```
LogLevel warn
ErrorLog /var/log/apache2/error.log
CustomLog /var/log/apache2/access.log combined
```

LogLevel warn: Este parámetro define el **nivel de detalle** de los mensajes que Apache registra en los logs de error.

Nivel	Descripción
emerg	Situaciones críticas que hacen que el servidor deje de funcionar.
alert	Eventos graves que requieren atención inmediata.
crit	Errores críticos en el servidor.
error	Errores generales que no detienen el servidor.
warn	Advertencias que pueden indicar problemas futuros. (Valor recomendado en producción)
notice	Mensajes informativos importantes.
info	Información detallada sobre la operación del servidor.
debug	Información muy detallada para depuración.
trace1 - trace8	Nivel de depuración más detallado (usado para desarrolladores).

Si se cambia *LogLevel warn* a *LogLevel debug*, Apache registrará más detalles útiles para diagnóstico, pero puede generar archivos de log muy grandes.

ErrorLog `${APACHE_LOG_DIR}/error.log`: Define la ubicación donde Apache guarda los **logs de errores**.

```
ErrorLog /var/log/apache2/error.log
```

Posibilidades adicionales: Se puede redirigir los logs de error a un *archivo específico* o incluso a *syslog*:

```
ErrorLog "|/usr/bin/logger -t apache_error"
```

Esto enviaría los logs al sistema de *logging* de Linux.

CustomLog $\${APACHE_LOG_DIR}/access.log$ combined: Define la ubicación y el **formato** del archivo donde se registran las solicitudes de acceso al servidor.

```
CustomLog /var/log/apache2/access.log combined
```

Formato	Descripción
combined	Formato extendido con más información (IP, fecha, agente de usuario, etc.). (Recomendado)
common	Formato básico sin detalles de referer ni agente de usuario.
vhost_combined	Similar a combined, pero muestra el VirtualHost asociado.
json	Formato JSON estructurado.
custom	Puedes definir tu propio formato personalizado.

Ejemplo de un log en formato combined:

```
192.168.1.10 - - [28/Feb/2025:12:34:56 +0000] "GET /index.html HTTP/1.1" 200 1024
"https://ejemplo.com" "Mozilla/5.0"
```

Ejemplo de un log en formato json:

```
CustomLog /var/log/apache2/access.log "{ \"ip\": \"%h\", \"time\": \"%t\", \"request\": \"%r\", \"status\": %>s, \"size\": %b }" json
```

Salida:

```
{ "ip": "192.168.1.10", "time": "[28/Feb/2025:12:34:56 +0000]", "request": "GET /index.html HTTP/1.1", "status": 200, "size": 1024 }
```

Reiniciar el servidor web cada vez que se modifique la configuración:

```
sudo systemctl restart apache2
```

Detectar intentos de ataque en los logs

Monitorear logs en tiempo real con *tail -f*:

```
tail -f /var/log/apache2/access.log
tail -n 50 /var/log/apache2/error.log
```

Mostrar los logs con paginación (para leer cómodamente)

Si el archivo es muy grande *less* o *more*:

```
less /var/log/apache2/access.log
```

Buscar accesos a la página de administración

Detectar intentos de acceso a rutas sensibles como */admin*:

```
grep "admin" /var/log/apache2/access.log
```

Buscar intentos de ejecución de comandos en la URL

Detectar posibles ataques de **inyección de comandos**:

```
grep "cmd=" /var/log/apache2/access.log
```

Buscar todas las peticiones de un usuario específico (IP)

Si se quiere revisar todas las solicitudes realizadas por una IP sospechosa (ej. 192.168.1.100):

```
grep "192.168.1.100" /var/log/apache2/access.log
```

Buscar intentos de inyección SQL en la URL

Analizar si hay consultas maliciosas en las peticiones:

```
grep -E "SELECT|INSERT|UPDATE|DELETE|DROP|UNION" /var/log/apache2/access.log
```

Ver errores HTTP 404 (páginas no encontradas)

Útil para detectar accesos a rutas inexistentes (*posible escaneo de vulnerabilidades*):

```
grep " 404 " /var/log/apache2/access.log
```

```
(root@kali)~# grep " 404 " /var/log/apache2/access.log
127.0.0.1 - - [28/Feb/2025:05:34:31 -0500] "GET /favicon.ico HTTP/1.1" 404 487 "http://localhost/csp/index.html" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:05:56:00 -0500] "GET /favicon.ico HTTP/1.1" 404 487 "http://localhost/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:05:59:30 -0500] "GET /favicon.ico HTTP/1.1" 404 487 "http://localhost/csp/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:06:00:32 -0500] "GET /favicon.ico HTTP/1.1" 404 487 "http://localhost/csp/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:06:05:26 -0500] "GET /favicon.ico HTTP/1.1" 404 397 "http://localhost/csp/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:06:08:43 -0500] "GET /favicon.ico HTTP/1.1" 404 396 "http://localhost/csp/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:06:16:13 -0500] "GET /favicon.ico HTTP/1.1" 404 397 "http://localhost/xss.php?xss=%3Cscript%3Ealert(%27XSS%20ejecutado!%27);%3C/script%3E" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:06:29:51 -0500] "GET /favicon.ico HTTP/1.1" 404 396 "http://localhost/csp/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:06:32:48 -0500] "GET /favicon.ico HTTP/1.1" 404 396 "http://localhost/csp/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [28/Feb/2025:09:49:52 -0500] "GET /favicon.ico HTTP/1.1" 404 396 "http://localhost/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
::1 - - [28/Feb/2025:10:24:05 -0500] "HEAD /index.php HTTP/1.1" 404 154 "-" "curl/8.12.1"
```

Filtrar por fecha específica

Ejemplo: Ver accesos del **28 de febrero de 2025**:

```
grep "28/Feb/2025" /var/log/apache2/access.log
```

Detectar ataques de fuerza bruta

Ver intentos de acceso fallidos en */login*:

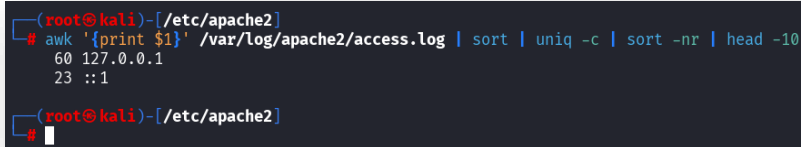
```
grep "/login" /var/log/apache2/access.log | grep " 401 "
```

Contar cuántas veces una IP ha intentado acceder

Para ver si una IP está haciendo muchas peticiones sospechosas:

```
awk '{print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -nr | head -10
```

Esto mostrará las **10 IPs con más solicitudes**.



```
(root@kali)~[/etc/apache2]
# awk '{print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -nr | head -10
60 127.0.0.1
23 ::1
(root@kali)~[/etc/apache2]
```

Buscar User-Agents sospechosos

Identificar *bots* o *scrapers* accediendo al servidor:

```
grep -E "bot|crawler|spider" /var/log/apache2/access.log
```

Ver actividad en tiempo real y filtrar por palabra clave

Monitorear *en vivo* cualquier intento de acceso a */wp-admin* (Página de administración de WordPress por defecto):

```
tail -f /var/log/apache2/access.log | grep "wp-admin"
```

Mitigación y Mejores Prácticas

Implementar medidas de seguridad adecuadas puede ayudar a **prevenir ataques** y mejorar la **respuesta ante incidentes**.

* Configurar alertas automáticas con Fail2Ban

¿Qué es Fail2Ban?

Fail2Ban es una herramienta que **monitorea los logs** en busca de patrones sospechosos (*como múltiples intentos fallidos de login*) y **bloquea automáticamente** la IP atacante *mediante reglas de firewall*.

1. Instalación de Fail2Ban (*Ubuntu/Debian*)

```
sudo apt update && sudo apt install fail2ban -y
```

2. Configurar **Fail2Ban** para Apache

Crear una copia del archivo de configuración predeterminado

```
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

Abrir el archivo de configuración:

```
sudo nano /etc/fail2ban/jail.local
```

Buscar [apache-auth] y añadir la siguiente configuración para proteger Apache:

```
[apache-auth]
enabled = true
port = http,https
logpath = /var/log/apache2/error.log
maxretry = 5
bantime = 3600
```

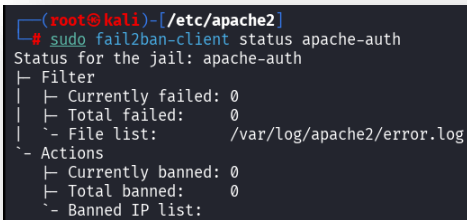
Esto bloqueará una IP por **1 hora** si realiza más de **5 intentos fallidos**.

3. Reiniciar **Fail2Ban** para aplicar cambios

```
sudo systemctl restart fail2ban
sudo systemctl enable fail2ban
```

4. Ver las IPs bloqueadas

```
sudo fail2ban-client status apache-auth
```



```
(root@kali)-[/etc/apache2]
# sudo fail2ban-client status apache-auth
Status for the jail: apache-auth
- Filter
| - Currently failed: 0
| - Total failed: 0
| - File list: /var/log/apache2/error.log
- Actions
| - Currently banned: 0
| - Total banned: 0
| - Banned IP list:
```

5. Ver reglas de iptables creadas por Fail2Ban

```
sudo iptables -L -n --line-numbers
```

6. Desbloquear una IP manualmente

```
sudo fail2ban-client unban <IP>
```

7. Bloquear una IP manualmente

```
sudo fail2ban-client set apache-auth banip <IP>
```

Ejercicio: Probar a bloquear una IP (9.9.9.9), revisar las reglas iptables creadas, así como las IP bloqueadas y desbloquear la IP

* Usar herramientas de SIEM (Security Information and Event Management)

¿Qué es un SIEM?

Un **SIEM** recopila, analiza y correlaciona eventos de seguridad desde múltiples fuentes, incluyendo logs de Apache, bases de datos y firewall, permitiendo una mejor detección de amenazas.

SIEM Populares

Herramienta	Descripción
ELK Stack (Elasticsearch, Logstash, Kibana)	Sistema Open Source para recolectar y visualizar logs. Ideal para Apache.
Splunk	Potente solución comercial para análisis de eventos de seguridad.
Wazuh	SIEM gratuito basado en OSSEC con integración en Elasticsearch.
Graylog	Alternativa de código abierto con buenas capacidades de análisis.



Configurar ELK Stack para Apache

ELK Stack (Elasticsearch, Logstash, Kibana) es una solución poderosa para monitoreo y análisis de logs en tiempo real. Integrarlo con Apache permite visualizar métricas de tráfico, detectar anomalías y mejorar la seguridad.

- *Elasticsearch*: Almacena y permite búsquedas en los logs de Apache.
- *Logstash*: Procesa y envía los logs de Apache a Elasticsearch.
- *Kibana*: Proporciona dashboards interactivos para visualizar logs y métricas de Apache.

1. Agregar el repositorio de Elastic

Ejecutar los siguientes comandos:

```
curl -fsSL https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elastic-keyring.gpg
```

```
echo "deb [signed-by=/usr/share/keyrings/elastic-keyring.gpg]
https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee
/etc/apt/sources.list.d/elastic-8.x.list
```

Nota: Si se desea una versión específica, cambiar 8.x por la versión deseada.

2. Actualizar los repositorios e instalar *Elasticsearch*, *Logstash* y *Kibana*

```
sudo apt update
sudo apt install elasticsearch logstash kibana -y
```

3. Iniciar y habilitar los servicios

```
sudo systemctl enable --now elasticsearch logstash kibana
```

4. Configurar *Elasticsearch*

Editar configuración de Elasticsearch

```
sudo nano /etc/elasticsearch/elasticsearch.yml
```

Modificar el fichero y lo dejamos como a continuación:

```
# ===== Elasticsearch Configuration =====
# ----- Paths -----
path.data: /var/lib/elasticsearch
path.logs: /var/log/elasticsearch
# ----- Network -----
network.host: 0.0.0.0
http.port: 9200
# ----- Discovery -----
discovery.type: single-node
# ----- Security -----
xpack.security.enabled: false
xpack.security.http.ssl.enabled: false
xpack.security.transport.ssl.enabled: false
```

Reiniciar *Elasticsearch*

```
sudo systemctl restart elasticsearch
```

Verificar si *Elasticsearch* está funcionando

```
curl -X GET "http://localhost:9200"
```

Si funciona correctamente, se debería ver una respuesta con información del servidor.

```
(root@kali)-[/home/kali]
# curl -X GET "http://localhost:9200"
{
  "name" : "kali",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "yeYfajMPTBSckT4V5XHBeQ",
  "version" : {
    "number" : "8.17.3",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "a091390de485bd4b127884f7e565c0cad59b10d2",
    "build_date" : "2025-02-28T10:07:26.089129809Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.0",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

5. Configurar Logstash para Apache

Crear un archivo de configuración en Logstash

```
sudo nano /etc/logstash/conf.d/apache.conf
```

Añadir la configuración para procesar logs de Apache

```
input {
  file {
    path => "/var/log/apache2/access.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
}

filter {
  grok {
    match => { "message" => "%{COMBINEDAPACHELOG}" }
  }
}

output {
  elasticsearch {
    hosts => ["http://localhost:9200"]
    index => "apache-logs"
  }
  stdout { codec => rubydebug }
}
```

Reiniciar Logstash

```
sudo systemctl restart logstash
```

Verificar que Logstash está procesando los logs

```
sudo journalctl -u logstash --no-pager | tail -n 20
```

```
Mar 11 16:20:06 kali systemd[1]: logstash.service: Main process exited, code=exited, status=1/FAILURE
Mar 11 16:20:06 kali systemd[1]: logstash.service: Failed with result 'exit-code'.
Mar 11 16:20:06 kali systemd[1]: logstash.service: Consumed 57.294s CPU time, 537.8M memory peak.
Mar 11 16:20:06 kali systemd[1]: logstash.service: Scheduled restart job, restart counter is at 72.
Mar 11 16:20:06 kali systemd[1]: Started logstash.service - logstash.
Mar 11 16:20:06 kali logstash[22723]: Using bundled JDK: /usr/share/logstash/jdk
Mar 11 16:20:23 kali logstash[22723]: Sending logstash logs to /var/log/logstash which is now configured via log4j2.properties
Mar 11 16:20:23 kali logstash[22723]: [2025-03-11T16:20:23.984][INFO ][logstash.runner ] Log4j configuration path used is: /etc/lo
gstash/log4j2.properties
Mar 11 16:20:23 kali logstash[22723]: [2025-03-11T16:20:23.993][INFO ][logstash.runner ] Starting Logstash {"logstash.version":"8
.17.2", "ruby.version":"ruby 9.4.9.0 (3.1.4) 2024-11-04 547c0b150e OpenJDK 64-Bit Server VM 21.0.6+7-LTS on 21.0.6+7-LTS +indy +jit [x86
_64-linux]}
Mar 11 16:20:23 kali logstash[22723]: [2025-03-11T16:20:23.996][INFO ][logstash.runner ] JVM bootstrap flags: [-Xms1g, -Xmx1g, -Dj
ava.awt.headless=true, -Dfile.encoding=UTF-8, -Djdk.compile.inlinedynamic=true, -XX:+HeapDumpOnOutOfMemoryError, -Djava.security.egd=file
:/dev/urandom, -Dlog4j2.isThreadContextMapInheritable=true, -Dlogstash.jackson.stream-read-constraints.max-string-length=200000000, -Dlogst
ash.jackson.stream-read-constraints.max-number-length=10000, -Djdk.io.File.enableADS=true, --add-exports=jdk.compiler/com.sun.tools.javac.api=ALL-UNNAMED, --add-exports=jdk.compiler/com.sun.tools.javac.file=ALL-UNNAMED, --add-exports=jdk.compiler/com.sun.tools.javac.parser=ALL-UNNAMED, --add-exports=jdk.compiler/com.sun.tools.javac.tree=ALL-UNNAMED, --add-exports=jdk.compiler/com.sun.tools.javac.util=ALL-UNNAMED, --add-opens=java.base/java.security=ALL-UNNAMED, --add-opens=java.base/java.io=ALL-UNNAMED, --add-opens=java.base/java.nio.channels=ALL-UNNAMED, --add-opens=java.base/sun.nio.ch=ALL-UNNAMED, --add-opens=java.management/sun.management=ALL-UNNAMED, -Dio.netty allocator=malloc]
Mar 11 16:20:24 kali logstash[22723]: [2025-03-11T16:20:24.040][INFO ][org.logstash.jackson.StreamReadConstraintsUtil] Jackson default valu
e override 'logstash.jackson.stream-read-constraints.max-string-length' configured to '200000000'
Mar 11 16:20:24 kali logstash[22723]: [2025-03-11T16:20:24.040][INFO ][org.logstash.jackson.StreamReadConstraintsUtil] Jackson default valu
e override 'logstash.jackson.stream-read-constraints.max-number-length' configured to '10000'
Mar 11 16:20:24 kali systemd[1]: Stopping logstash.service - logstash.
Mar 11 16:20:24 kali systemd[1]: logstash.service: Main process exited, code=exited, status=143/n/a
Mar 11 16:20:24 kali systemd[1]: logstash.service: Failed with result 'exit-code'.
Mar 11 16:20:24 kali systemd[1]: Stopped logstash.service - logstash.
Mar 11 16:20:24 kali systemd[1]: logstash.service: Consumed 54.454s CPU time, 597.9M memory peak.
Mar 11 16:20:24 kali systemd[1]: Started logstash.service - logstash.
Mar 11 16:20:24 kali logstash[22935]: Using bundled JDK: /usr/share/logstash/jdk
root@kali: ~ /home/kali
```

6. Configurar Kibana

Editar la configuración de Kibana

```
sudo nano /etc/kibana/kibana.yml
```

Activar las siguientes líneas

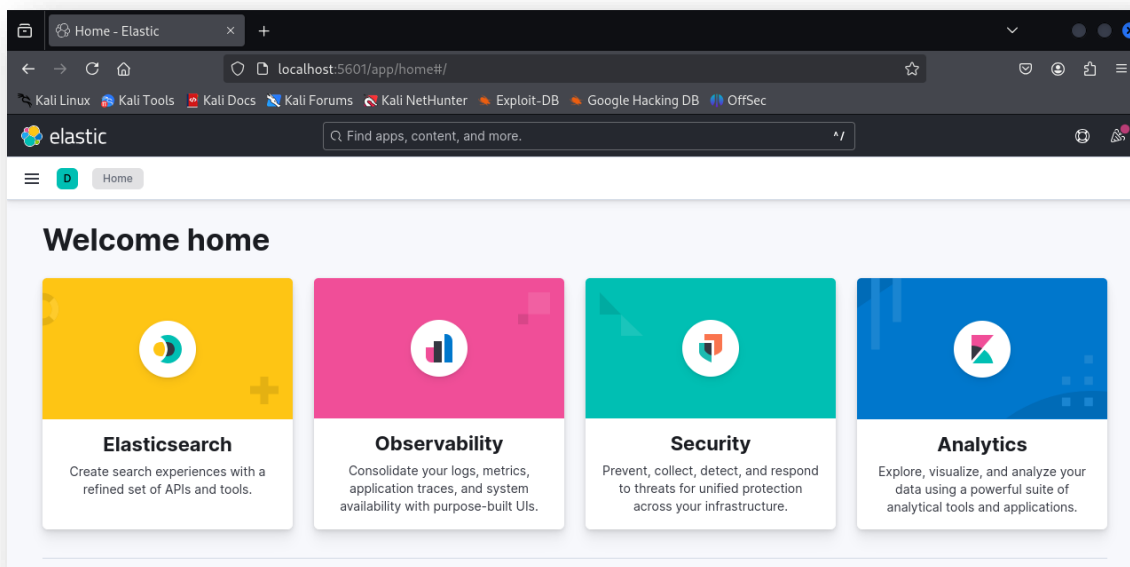
```
server.port: 5601
server.host: "0.0.0.0" # Permite acceso remoto (opcional)
elasticsearch.hosts: ["http://localhost:9200"]
```

Reiniciar Kibana

```
sudo systemctl restart kibana
```

Acceder a Kibana desde el navegador

```
http://localhost:5601
```



7. Instalar y configurar *Filebeat* para Apache

Instalar Filebeat

```
sudo apt install filebeat -y
```

Habilitar el módulo de Apache en Filebeat

```
sudo filebeat modules enable apache
```

Editar la configuración de Filebeat

```
sudo nano /etc/filebeat/filebeat.yml
```

Modificar la salida a Elasticsearch

```
output.elasticsearch:  
  hosts: ["localhost:9200"]
```

Reiniciar Filebeat

```
sudo systemctl restart filebeat
```

Editar el archivo de configuración del módulo Apache:

```
sudo nano /etc/filebeat/modules.d/apache.yml
```

Cambia *enabled: false* a *enabled: true* en ambas secciones y añade las *var.paths*:

```
access:
  enabled: true
  var.paths: ["/var/log/apache2/access.log*"]

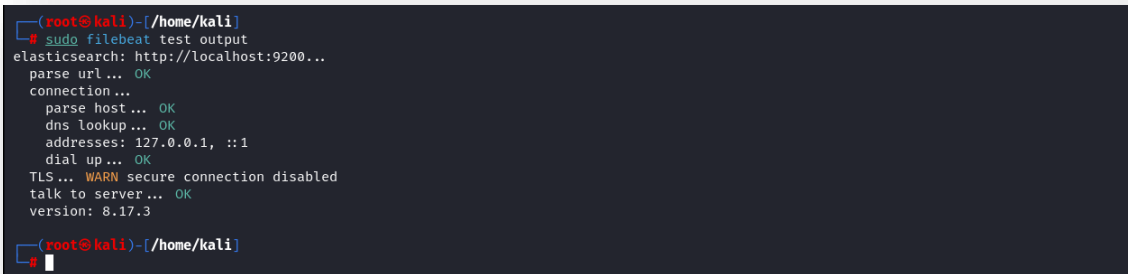
error:
  enabled: true
  var.paths: ["/var/log/apache2/error.log*"]
```

Configurar Filebeat para recolectar logs de Apache

```
sudo filebeat setup
```

Verificar que Filebeat está enviando logs

```
sudo filebeat test output
```



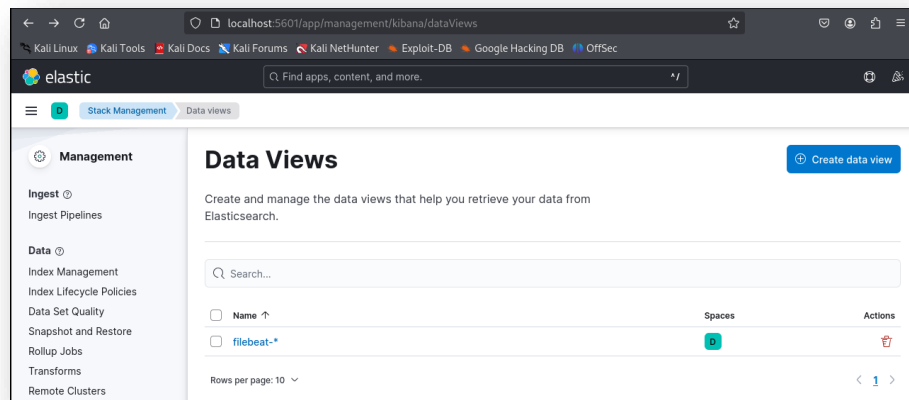
```
(root@kali)-[/home/kali]
# sudo filebeat test output
elasticsearch: http://localhost:9200...
parse url... OK
connection...
parse host... OK
dns lookup... OK
addresses: 127.0.0.1, ::1
dial up... OK
TLS... WARN secure connection disabled
talk to server... OK
version: 8.17.3
(root@kali)-[/home/kali]
```

8. Verificar que los logs de Apache están en Kibana

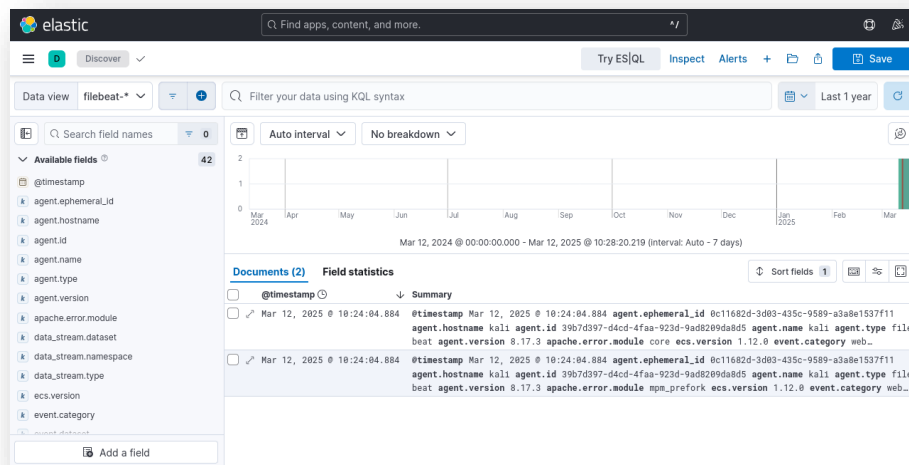
Acceder a Kibana desde el navegador

```
http://localhost:5601
```

En el menú lateral "Management" seleccionar **"Stack Management"** → **"Data Views"**.



Si todo está configurado correctamente ir a la sección **Analytics** → **Discover**, buscar el índice *filebeat-** y se debería ver los *logs* en tiempo real.



ELK + Filebeat

- **Elasticsearch:** Almacena y permite buscar logs de Apache
- **Logstash:** Procesa los logs antes de enviarlos a Elasticsearch
- **Kibana (Analytics):** Visualiza los logs en dashboards interactivos
- **Filebeat:** Recoge y envía logs de Apache

Resumen

- **Fail2Ban** es una excelente primera línea de defensa contra ataques automatizados.
- **SIEMs** como **ELK** o **Wazuh** permiten un análisis profundo y detección temprana de amenazas.
- **Monitorear logs en tiempo real y configurar alertas** mejora la seguridad y reduce tiempos de respuesta ante incidentes.